

DETECÇÃO DE TALHERES COM DADOS SINTÉTICOS GERADOS NO BLENDER

Pipeline completo de geração, anotação automática, treinamento e avaliação com YOLOv8

Projeto	Informação
Autor	Adriano César Santana
Curso	Fastcamp de Dados Sintéticos para IA e Visão Computacional
Problema	Detecção de garfo, faca e colher
Dataset	100 imagens sintéticas anotadas automaticamente
Modelo	YOLOv8n com transferência de aprendizado
Ano	2026

RELATÓRIO TÉCNICO

Resumo

Este trabalho apresenta um pipeline completo para geração de dados sintéticos e treinamento de um modelo de visão computacional destinado à detecção de três classes de talheres: garfo, faca e colher. Os objetos foram representados por modelos 3D simples e organizados em uma cena no Blender. Um script em Python, utilizando a API bpy, automatizou a variação da posição, rotação e escala dos objetos, da iluminação e da cor do fundo, além da renderização e da geração das caixas delimitadoras no formato YOLO.

Foram produzidas 100 imagens sintéticas de 640 x 640 pixels, divididas em 70 imagens para treinamento, 20 para validação e 10 para teste. Um modelo YOLOv8n pré-treinado foi ajustado durante 50 épocas em GPU no Google Colab. Na validação sintética final, o modelo apresentou precisão de 0,9902, revocação de 1,0000, mAP@50 de 0,9950 e mAP@50-95 de 0,8892. Embora os resultados demonstrem a viabilidade do pipeline, eles devem ser interpretados dentro do domínio sintético, pois ainda não foi conduzida uma avaliação sistemática com fotografias reais.

Palavras-chave

Dados sintéticos; Blender; visão computacional; detecção de objetos; YOLO; anotação automática.

1. Introdução

Modelos de visão computacional dependem de conjuntos de dados representativos e corretamente anotados. A coleta manual de imagens e a construção de caixas delimitadoras demandam tempo, recursos e conferência humana. A geração sintética oferece uma alternativa controlável, na qual a cena, os objetos e as condições de captura são conhecidos e podem ser modificados programaticamente.

O projeto integra modelagem 3D, scripting no Blender, renderização, anotação automática, organização do dataset, treinamento de rede neural e análise crítica dos resultados. O escopo foi dimensionado para uma implementação funcional em um dia, sem eliminar as entregas técnicas previstas no desafio final.

2. Definição do problema e escopo

2.1 Problema de visão computacional

O problema escolhido foi a detecção de objetos. Dada uma imagem contendo utensílios, o modelo deve localizar cada item por meio de uma caixa delimitadora e classificá-lo como garfo, faca ou colher. Essa aplicação foi selecionada por utilizar objetos cotidianos, possuir três classes visualmente distintas e permitir a criação de uma cena 3D controlada.

2.2 Especificação do dataset

Elemento	Especificação
Quantidade	100 imagens sintéticas
Resolução	640 x 640 pixels
Classes	garfo, faca e colher
Objetos por imagem	3
Anotações	Caixas delimitadoras no formato YOLO
Variações	posição, rotação, escala, fundo e iluminação
Divisão	70% treino, 20% validação e 10% teste

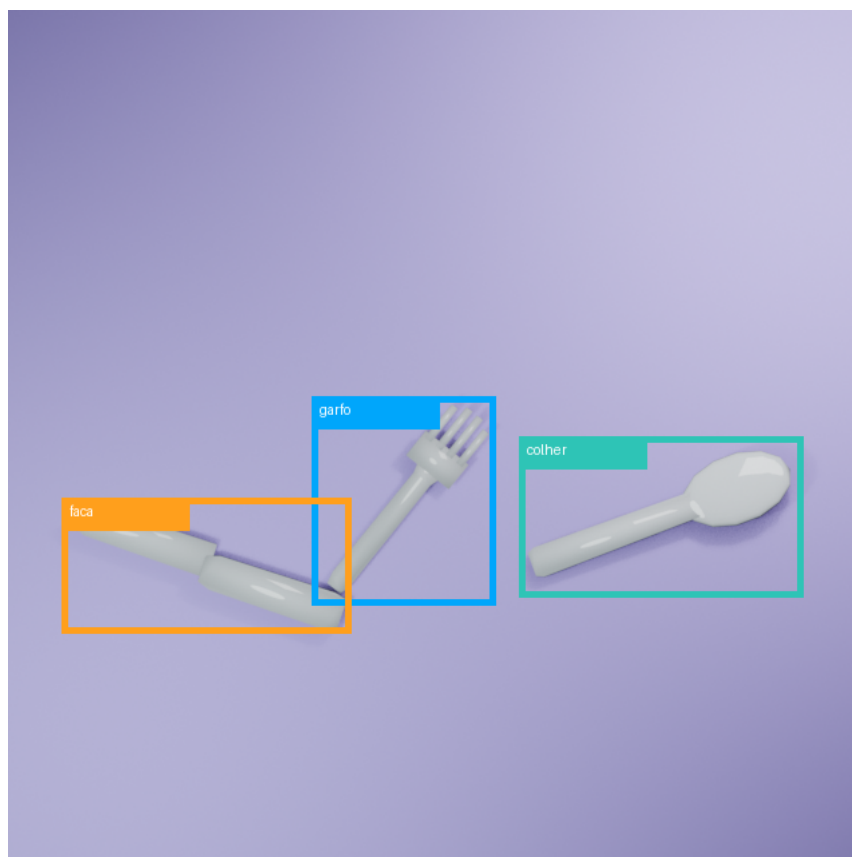


Figura 1 - Exemplo de imagem sintética e caixas delimitadoras geradas automaticamente.

3. Preparação da cena no Blender

Foram utilizados modelos 3D simples de garfo, faca e colher no formato FBX. Os modelos foram importados para o Blender e mantidos como objetos independentes, identificados pelos nomes garfo, faca e colher. A cena final está disponível em `blender/Projeto_Final.blend`, enquanto os

modelos individuais se encontram em blender/modelos.

O script configura um plano horizontal de fundo, câmera ortográfica em vista superior e duas fontes de luz do tipo área. A visão superior reduz cortes e facilita a interpretação das caixas delimitadoras. Os objetos são normalizados para dimensões semelhantes e posicionados acima do plano para permitir a formação de sombras.

Componente	Configuração
Câmera	Ortográfica, vista superior, escala 8,2
Fundo	Plano horizontal com cor variável
Luz principal	Área, energia variável entre 650 e 1.250
Luz auxiliar	Área, energia variável entre 250 e 650
Renderização	EEVEE, PNG, RGB, 640 x 640

4. Automação e anotação

4.1 Geração programática

O arquivo `blender/gerar_dataset_talheres.py` implementa todo o processo. Em cada iteração, as posições-base das três classes são embaralhadas. Em seguida, são adicionados deslocamentos aleatórios, rotações em torno dos três eixos e variações uniformes de escala. A cor e a rugosidade do fundo, a energia das luzes e a luz ambiente também são modificadas.

```
QUANTIDADE_IMAGENS = 100
RESOLUCAO = 640
CLASSES = {"garfo": 0, "faca": 1, "colher": 2}
```

4.2 Anotação automática

Para cada objeto, os oito cantos da caixa delimitadora tridimensional são transformados pelas matrizes do objeto e projetados no plano da câmera. Os valores mínimo e máximo das coordenadas projetadas definem a caixa 2D. Por fim, centro, largura e altura são normalizados para o intervalo de zero a um e gravados no padrão YOLO.

```
classe centro_x centro_y largura altura
0 0.237829 0.548315 0.191257 0.335829
```

A automação assegurou correspondência de nomes entre imagens e rótulos. A inspeção final confirmou 100 imagens, 100 arquivos de anotação e três objetos anotados por imagem, totalizando 300 instâncias.

5. Organização do dataset

Conjunto	Imagens	Rótulos	Instâncias
Treinamento	70	70	210
Validação	20	20	60
Teste	10	10	30
Total	100	100	300

```
dataset/
■■■ train/images e train/labels
■■■ valid/images e valid/labels
■■■ test/images e test/labels
■■■ dataset.yaml
```

6. Configuração e treinamento da rede neural

Foi selecionada a arquitetura YOLOv8n, a menor variante da família YOLOv8. A escolha priorizou rapidez de treinamento, tamanho reduzido do arquivo de pesos e adequação a um dataset pequeno. O modelo foi inicializado com pesos pré-treinados e ajustado ao problema de três classes, caracterizando transferência de aprendizado.

Hiperparâmetro	Valor
Modelo	YOLOv8n pré-treinado
Épocas	50
Tamanho de entrada	640 x 640
Tamanho do lote	8
Otimizador	Automático, definido pela biblioteca
Dispositivo	GPU no Google Colab
Paciência	15 épocas
Precisão mista	Ativada

O notebook `treinamento/treinamento_yolo.ipynb` documenta instalação, carregamento, conferência do dataset, treinamento, avaliação, predição e exportação. O melhor peso foi preservado em `treinamento/best.pt`.

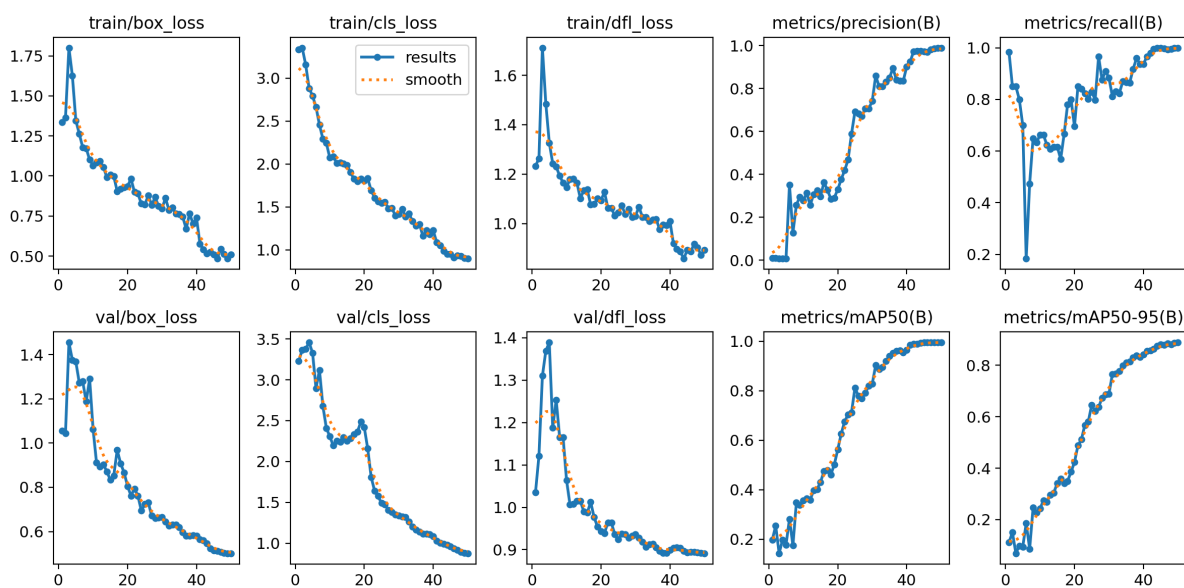


Figura 2 - Evolução das perdas e métricas durante as 50 épocas de treinamento.

7. Avaliação e resultados

As métricas consolidadas abaixo correspondem à validação sintética registrada ao final da 50ª época. O desempenho cresceu de maneira consistente durante o treinamento, alcançando alta precisão e revocação para as três classes.

Métrica	Resultado	Interpretação
Precisão	0,9902	Poucas detecções incorretas
Revocação	1,0000	Todos os objetos de validação foram recuperados
mAP@50	0,9950	Excelente desempenho com IoU 0,50
mAP@50-95	0,8892	Boa localização sob critérios mais rigorosos

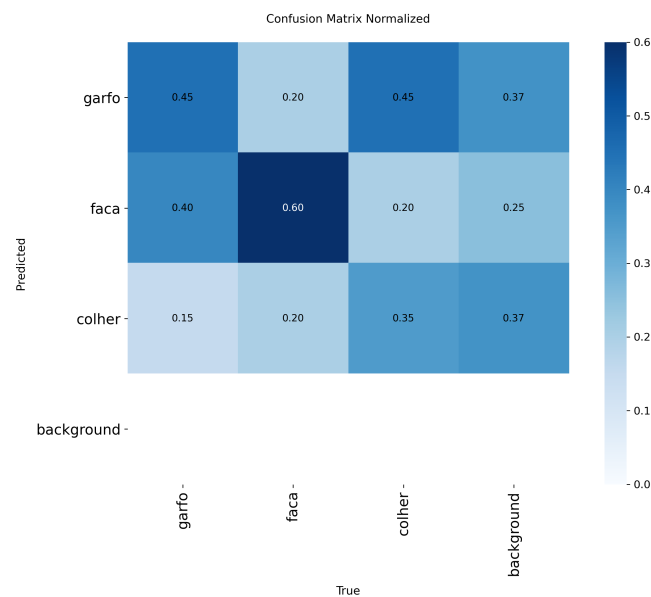


Figura 3 - Matriz de confusão normalizada do conjunto de validação.

A diferença entre mAP@50 e mAP@50-95 mostra que a classificação e a localização geral foram consistentes, mas ainda há espaço para melhorar o ajuste exato das caixas sob limiares elevados de IoU. Esse comportamento também pode refletir o uso das caixas 3D projetadas, que nem sempre acompanham exatamente o contorno visual do objeto.

7.1 Análise qualitativa



Figura 4 - Exemplos de detecções realizadas durante a validação.

As predições qualitativas mostram a identificação simultânea das três classes em diferentes orientações e fundos. As dez imagens reservadas para teste também foram processadas e estão disponíveis em resultados/predicoes.

7.2 Limitações e domain gap

A principal limitação é o domain gap entre as renderizações e fotografias reais. Os objetos 3D apresentam geometrias e materiais simplificados; os fundos são lisos; não foram incluídos ruído de sensor, desfoque significativo, texturas complexas, mãos, pratos, alimentos ou oclusões severas. Além disso, a câmera permaneceu predominantemente em vista superior e o dataset possui somente 100 imagens.

As métricas elevadas devem, portanto, ser entendidas como evidência de aprendizagem dentro do domínio sintético, e não como comprovação de generalização para o mundo real. Uma avaliação futura deve utilizar fotografias reais nunca vistas, seguida, se necessário, de ajuste fino com um pequeno conjunto real anotado.

8. Repositório, documentação e reprodução

O repositório reúne a cena Blender, os modelos FBX, o código de geração, o dataset, o notebook, o melhor peso e os resultados. O arquivo README.md apresenta a estrutura do projeto e os comandos essenciais.

```
git clone https://github.com/scadriano/AdrianoCesarSantana-Fastcamp_de_Dados_Sinteticos.git
cd AdrianoCesarSantana-Fastcamp_de_Dados_Sinteticos/Projeto\Final
pip install -r requirements.txt
```

8.1 Regerar o dataset

Abra blender/Projeto_Final.blend, carregue blender/gerar_dataset_talheres.py no editor de scripts e execute-o com Alt + P. O script cria as pastas e grava imagens e rótulos ao lado do arquivo .blend.

8.2 Treinar ou usar o modelo

O notebook pode ser executado no Google Colab com GPU. Para inferência local, instale as dependências e carregue treinamento/best.pt com a classe YOLO. O caminho de origem pode apontar para uma imagem, pasta ou vídeo.

9. Conclusão

O projeto cumpriu o objetivo de conceber e executar um pipeline completo de dados sintéticos para visão computacional. A cena foi configurada no Blender, os parâmetros foram variados programaticamente, as anotações foram produzidas sem marcação manual, o dataset foi organizado no padrão YOLO e um modelo YOLOv8n foi treinado e avaliado.

Os resultados obtidos no domínio sintético demonstram que um conjunto pequeno e controlado pode ser suficiente para validar tecnicamente o pipeline. A contribuição principal do trabalho não é apenas o valor das métricas, mas a integração reproduzível entre geração 3D, anotação, treinamento e documentação. A etapa seguinte mais importante consiste em medir o desempenho em imagens reais e reduzir o domain gap por meio de maior diversidade sintética e transferência de domínio.

10. Artefatos entregues

Requisito	Artefato
Cena configurada	blender/Projeto_Final.blend
Modelos 3D	blender/modelos/*.fbx
Automação e anotação	blender/gerar_dataset_talheres.py
Dataset organizado	dataset/train, valid e test
Configuração YOLO	dataset/dataset.yaml
Treinamento	treinamento/treinamento_yolo.ipynb
Modelo treinado	treinamento/best.pt
Métricas e gráficos	resultados/
Documentação	README.md e relatorio_tecnico.pdf

Referências

BLENDER FOUNDATION. Blender Python API Documentation. Disponível em: <https://docs.blender.org/api/current/>. Acesso em: 6 ago. 2026.

ULTRALYTICS. Ultralytics YOLO Documentation. Disponível em: <https://docs.ultralytics.com/>. Acesso em: 6 ago. 2026.

JOCHER, G.; CHAURASIA, A.; QIU, J. Ultralytics YOLO. 2023. Repositório de software disponível em: <https://github.com/ultralytics/ultralytics>.